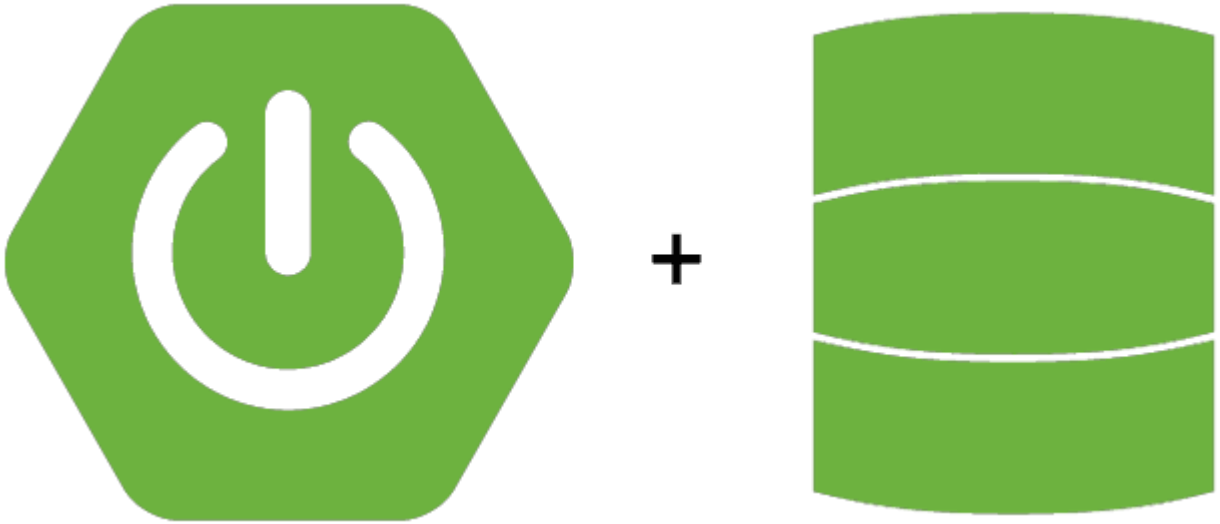
SPRING DATA JPA – PREMIÈRE ENTITÉ



Plan du Cours

- Persistance
- JDBC
- ORM
- JPA
- Hibernate
- Spring Data
- Spring Data JPA
- Entité
- Annotations

- Configuration d'une DataSource avec Spring Boot
- TP Spring Boot + Spring Data JPA : 1ère Entité

PERSISTANCE

- **Le Modèle Relationnel** : les systèmes de gestion de bases de données relationnels (**SGBDR**) sont devenus un pilier incontournable dans le développement d'applications.

- **Le modèle Object** : se base sur la programmation orientée objet. Celle-ci permet aux applications d'atteindre un excellent niveau de qualité et de flexibilité.

- Comment stocker les objets modélisés de la mémoire vers les SGBDR ?

- On parle ainsi de **persistance d'objets métiers**.

PERSISTANCE

- Pour persister les données :
  - Développer manuellement le code de projection des objets sur le support relationnel (Utilisation de JDBC natif).

  Ou :

  - Utiliser un Framework de projection objet-relationnel : EclipseLink, Hibernate , **Spring Data JPA** … Ces Frameworks implémentent la spécification JakartaEE JPA.

JDBC

- **JDBC** (Java DataBase Connectivity) est une interface de programmation créée par Sun Microsystems (racheté par Oracle Corporation).
- C'est une **API** (Application Programming Interface) pour les programmes utilisant la plateforme Java.
- Elle permet aux applications Java d'accéder à des bases de données, en utilisant des pilotes JDBC (Drivers).
- Des pilotes JDBC sont disponibles pour tous les systèmes connus de bases de données relationnelles.

JDBC

- L'utilisation de l'**API JDBC** était une solution pour les développeurs java pour manipuler les données dans une base de donnée SQL (CRUD).

- **Inconvénients** :
  - Pas de séparation entre le code technique et le code métier.
  - utilisation du langage SQL rend la couche d'accès aux données difficilement maintenable.

- Exemple :

JDBC

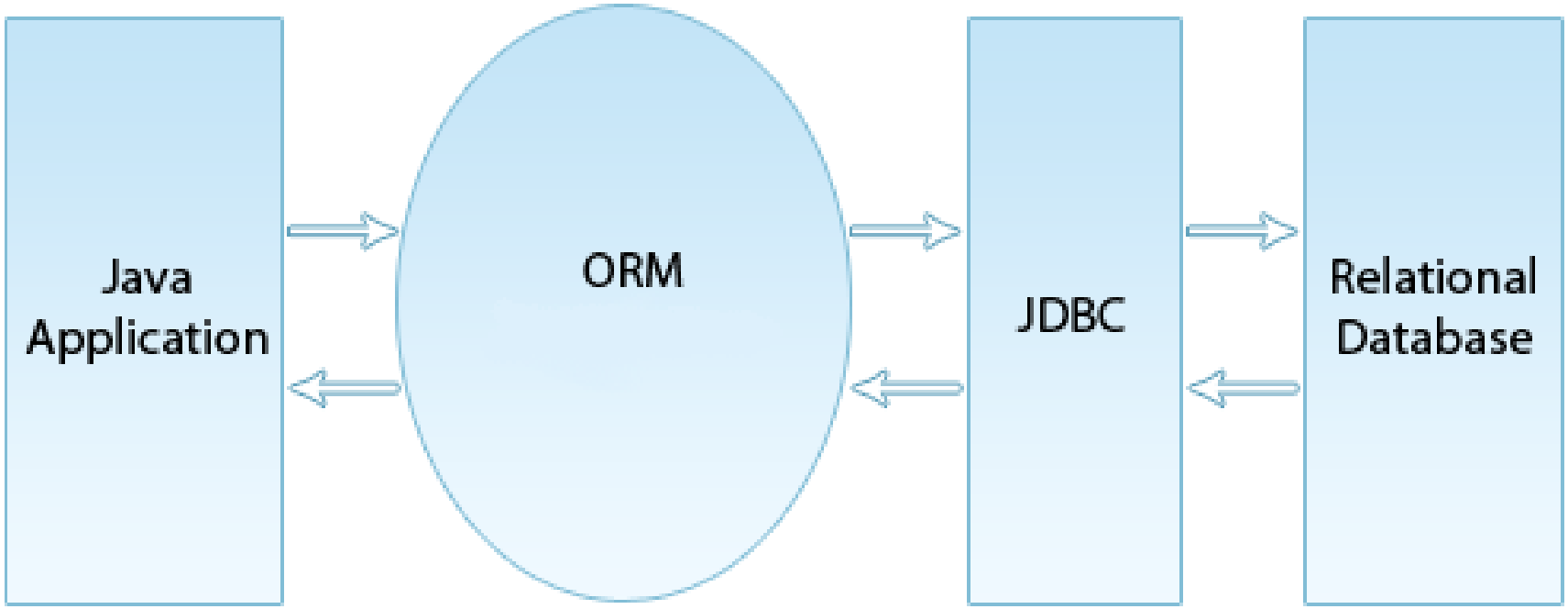
```
public void addPiste(Piste piste) {
    Connection conn = null; PreparedStatement preparedStmt = null;

    try {
        conn = DriverManager.getConnection(DB_URL,USER,PASSWORD);

        String insertSQL = "INSERT INTO Piste (idPiste, numPiste, nomPiste, couleur, longueur,
          pente) VALUES (?, ?, ?, ?, ?, ?, ?)";
        preparedStmt = conn.prepareStatement(insertSQL);
        preparedStmt.setLong(1, piste.getIdPiste());
        preparedStmt.setLong(2, piste.getNumPiste());
        preparedStmt.setString(3, piste.getNomPiste()));
        preparedStmt.setString(4, piste.getCouleur());
        preparedStmt.setInteger(5, piste.getLongeur());
        preparedStmt.setInteger(6, piste.getPente());
        preparedStmt.execute();
        }
    catch (SQLException e) { e.printStackTrace(); }
    finally { try { preparedStmt.close(); } catch (SQLException e) { e.printStackTrace(); }
}
```

ORM

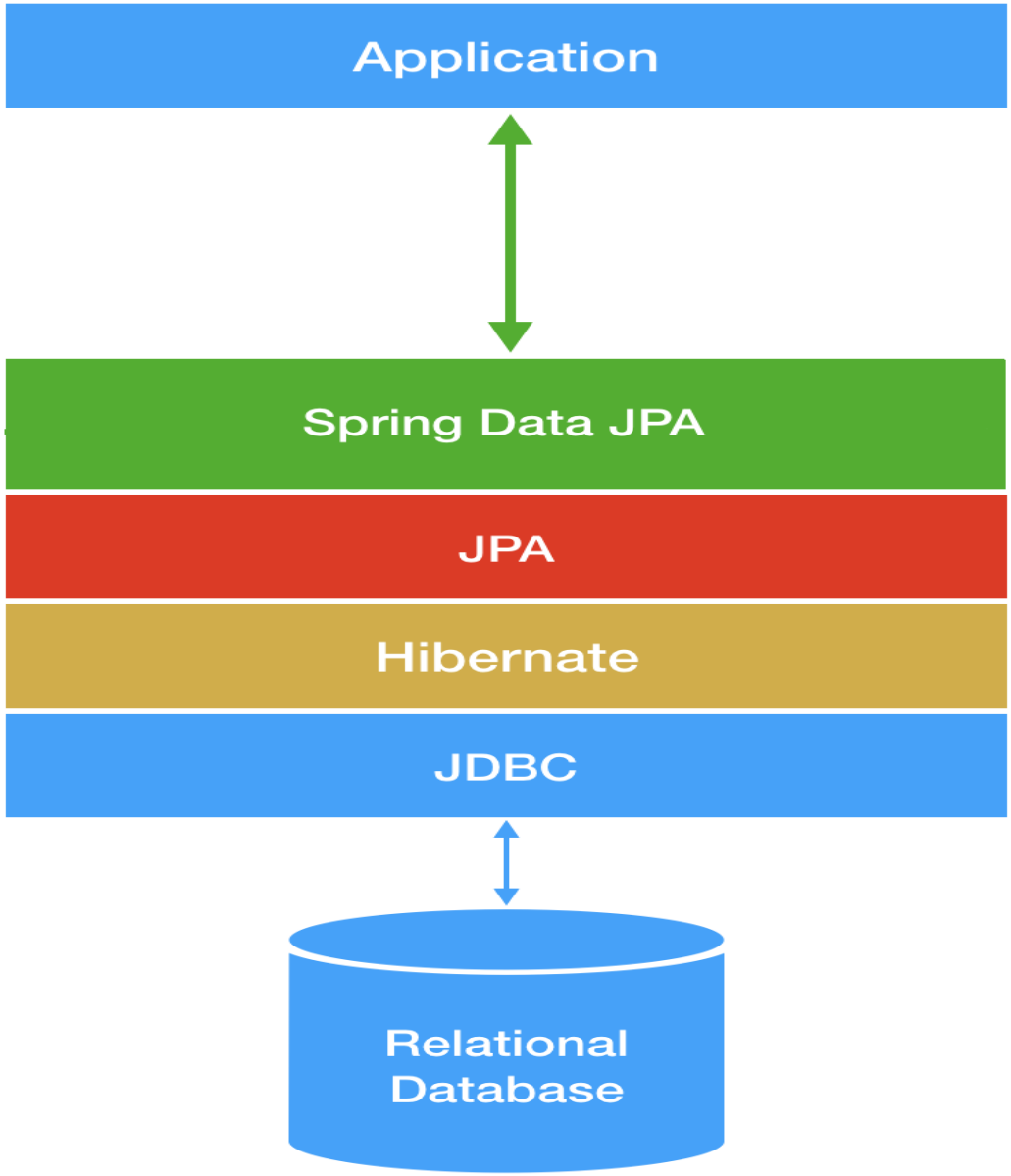
- Pour éviter les inconvénients liés à l'utilisation de l'API JDBC nativement, le concept d'**ORM** a vu le jour.
- Le **M**apping **O**bjet ←→ **R**elationnel (ORM) est une technique de programmation, qui permet d'associer une ou plusieurs classes avec une table, et chaque attribut de la classe avec un champ de la table.

- Elle vise à réduire la quantité de code produit par l'API JDBC (les opérations sont les mêmes : Connexion + CRUD + Déconnexion).

ORM - JDBC



ORM - JDBC



User.java
UserRepository.java

CrudRepository ou
PagingAndSortingRepository ou
JpaRepository

javax.persistence.EntityManager

org.hibernate.Session

javax.sql.PreparedStatement

JPA

- Pour normaliser le fonctionnement des ORM, **JPA** a été mis en place.

- La **Jakarta Persistance API** (abrégée en JPA), (anciennement Java Persistance API) est une interface de programmation Java permettant de normaliser l'utilisation et la communication avec la couche de données, d'une application Java.

JPA

- l'utilisation de **JPA** nécessite **un fournisseur de persistance (ORM)** qui implémente les spécifications JPA (Hibernate, Toplink, ...)

- Dans ce cours nous allons utiliser **Spring Data JPA** qui se base sur **Hibernate** comme implémentation de la spécification **JPA**.

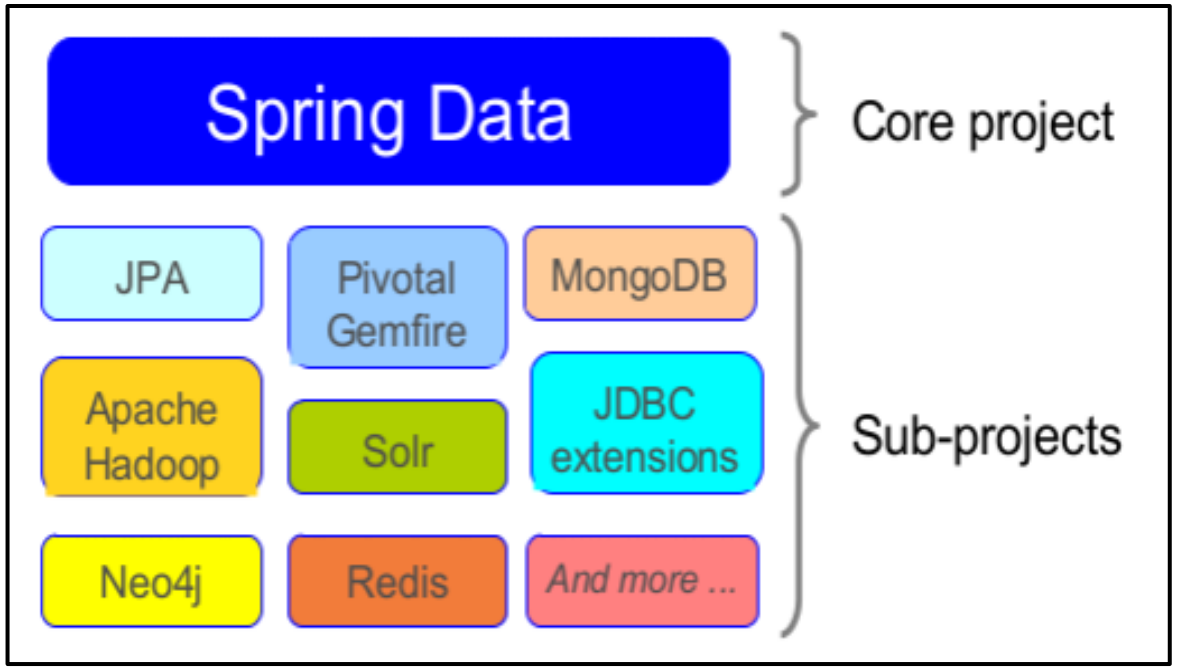
- **JPA** est une **spécification** (normalisation et standardisation de la communication avec la DB).
- **Hibernate** est un produit (**Implémentation** de cette spécification).

HIBERNATE

- **Hibernate** est un Framework open source gérant la persistance des objets en base de données relationnelle.
- Il gère le Mapping entre les objets de l'application et la base de données.
- C'est un projet maintenu par l'entreprise JBoss, appartenant à RedHat.
- L'utilisation de JPA / HIBERNATE nous permet de gagner du temps en développement :
  - Génération automatique du code SQL (CRUD).
  - Génération automatique des tables à partir des entités Java.

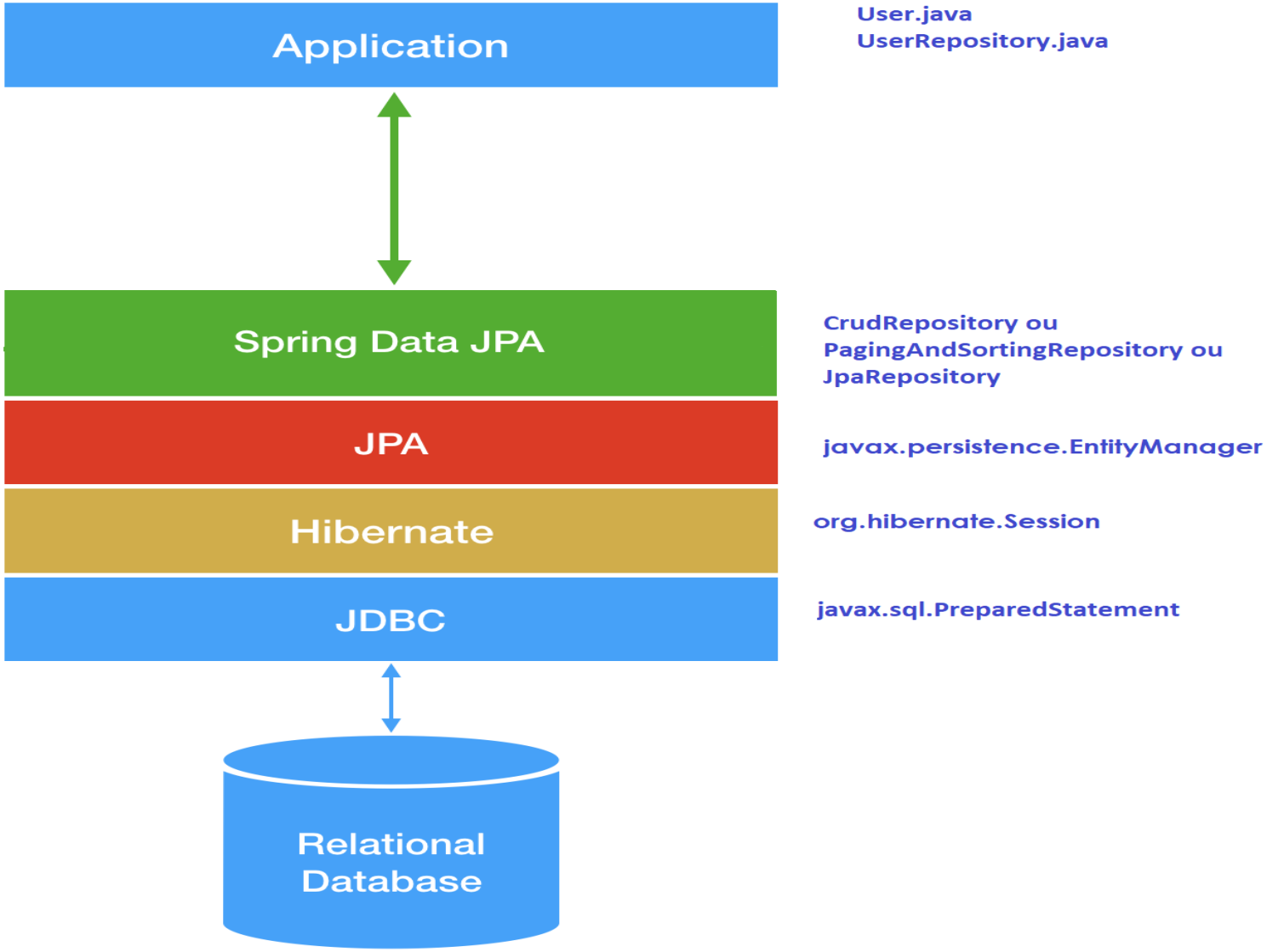
SPRING DATA

- C'est un module Spring (Pivotal) qui a pour but de :
  - Faciliter l'écriture des couches d'accès aux données.
  - Offrir une abstraction commune pour l'accès aux données quelle que soit la source de données (SQL ou NoSQL).



SPRING DATA JPA

- Spring Data JPA est un sous projet du projet Spring Data.



ENTITÉ JPA

- Une entité est une classe dont les instances peuvent être persistantes.
- Elle est déclarée avec l'annotation **@Entity**.
- Elle possède au moins une propriété déclarée comme identité de l'entité avec l'annotation **@Id**.

- Utilisation d'annotations
  - Sur la classe : correspondance avec la table associée.
  - Sur les attributs ou sur les propriétés : correspondance avec les colonnes de la table.
- La classe est un JavaBean (attributs, accesseurs, mutateurs)
- Exemple : (qui sera créé pas à pas par la suite) :

ENTITE JPA

```
package tn.esprit.tpfoyer.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
public class Chambre {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idChambre;

    private Long numeroChambre;

    @Enumerated(EnumType.STRING)
    private TypeChambre typeC;

}
```

ANNOTATIONS

- **@Entity** : **Obligatoire**, sur la classe.

- Une entité, déclarée par l'annotation **@Entity** définit une classe Java comme étant persistante et donc associée à une table dans la base de données.

- **@Table : Optionnel**. Par défaut, une entité est associée à la table portant le même nom que la classe. Il est possible d'indiquer le nom de la table par une annotation **@Table**.

- **@Table (name="nomTable")** : Facultatif, sur la classe. Cette annotation permet de mapper les objets de la classe avec la table dont le nom est redéfini. Si omis, la table prend le nom de la classe.

ANNOTATIONS

- **@Id** : La déclaration d'une clé primaire est obligatoire. Sur un attribut ou sur le getter

- **@GeneratedValue** : Facultatif, sur l'attribut ou sur le getter annoté avec @Id. Définit la manière dont la base gère la génération de la clé primaire. L'attribut "**strategy**" obligatoire pouvant avoir comme valeur : AUTO / IDENTITY / SEQUENCE / TABLE :

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long idChambre;
```

ANNOTATIONS

- **Strategy = GenerationType.AUTO** : la génération de la clé primaire est garantie par le fournisseur de persistance (hibernate), une séquence unique de clé primaire **pour tout le schéma de base de données** défini dans une table **hibernate_sequence**.

- **Strategy = GenerationType.TABLE** : la génération des clés primaires est garantie par le fournisseur de persistance (hibernate), une séquence de clés primaires **par table** définie dans une table **hibernate_sequences**, cette table contient deux colonnes : une pour le nom de la séquence et l'autre pour la prochaine valeur.

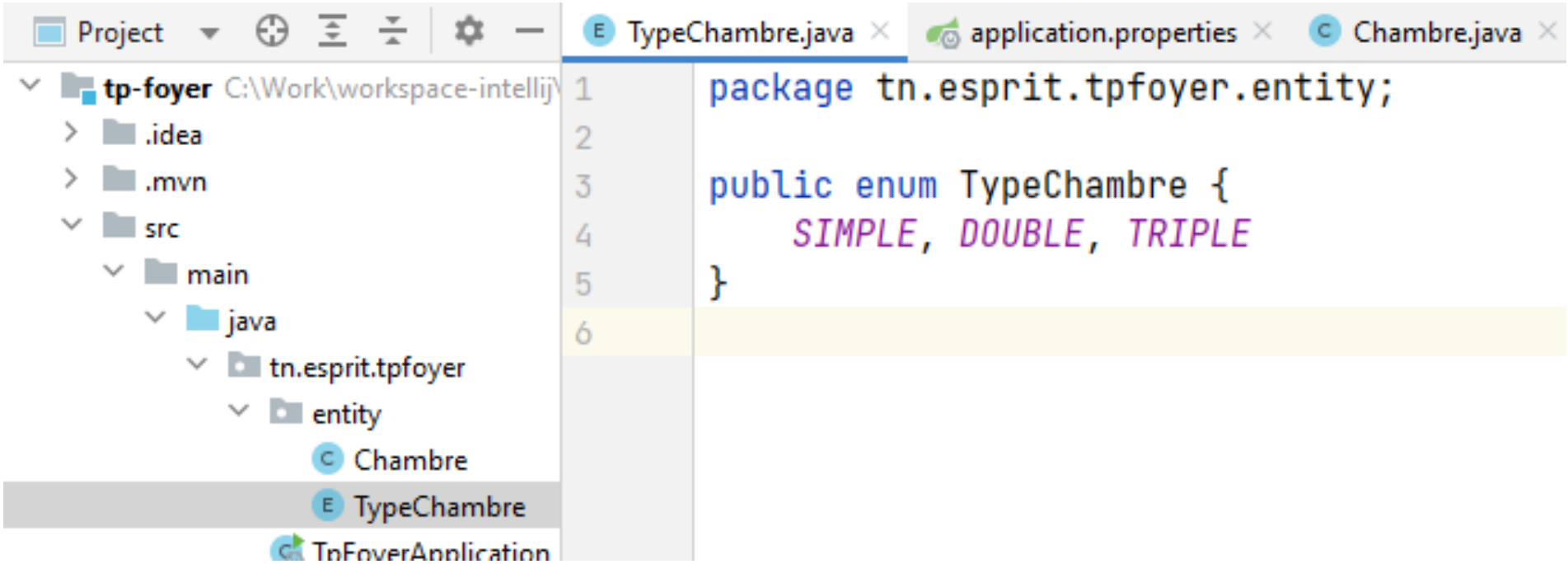
- **Strategy = GenerationType.IDENTITY**. Hibernate s'appuie alors sur le mécanisme propre au SGBD pour la production de l'identifiant.
  Dans le cas de MySQL, c'est l'option AUTO-INCREMENT, dans le cas de Postgres ou Oracle, c'est une séquence. **On recommande l'IDENTITY.**

- **Strategy = GenerationType.SEQUENCE** = GenerationType.AUTO, mais c'est nous qui définissions les détails de la séquence.

ANNOTATIONS

- **<u>Classe de type Enum</u> à créer** :

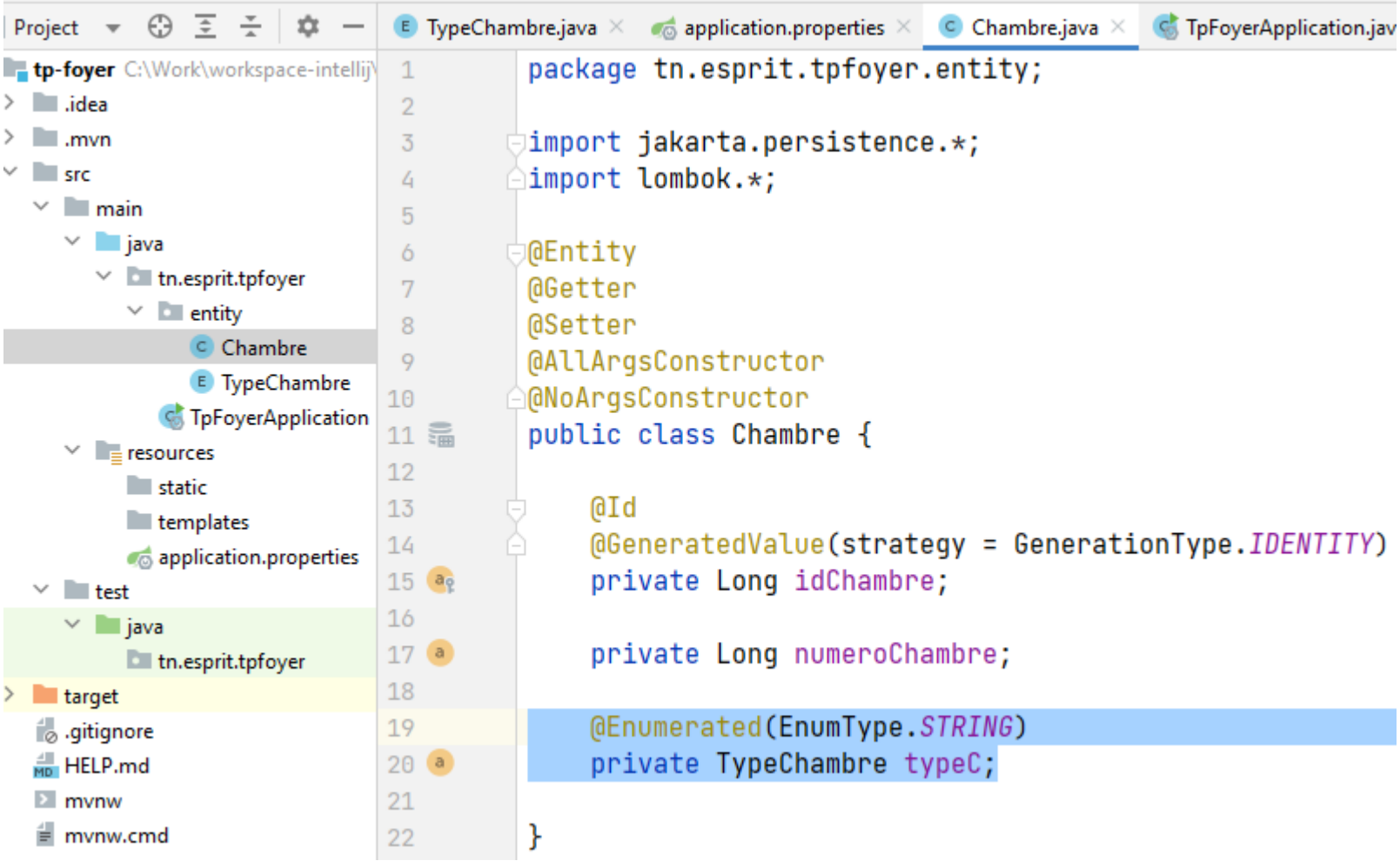
```
package tn.esprit.tpfoyer.entity;

public enum TypeChambre {
    SIMPLE, DOUBLE, TRIPLE
}
```



ANNOTATIONS

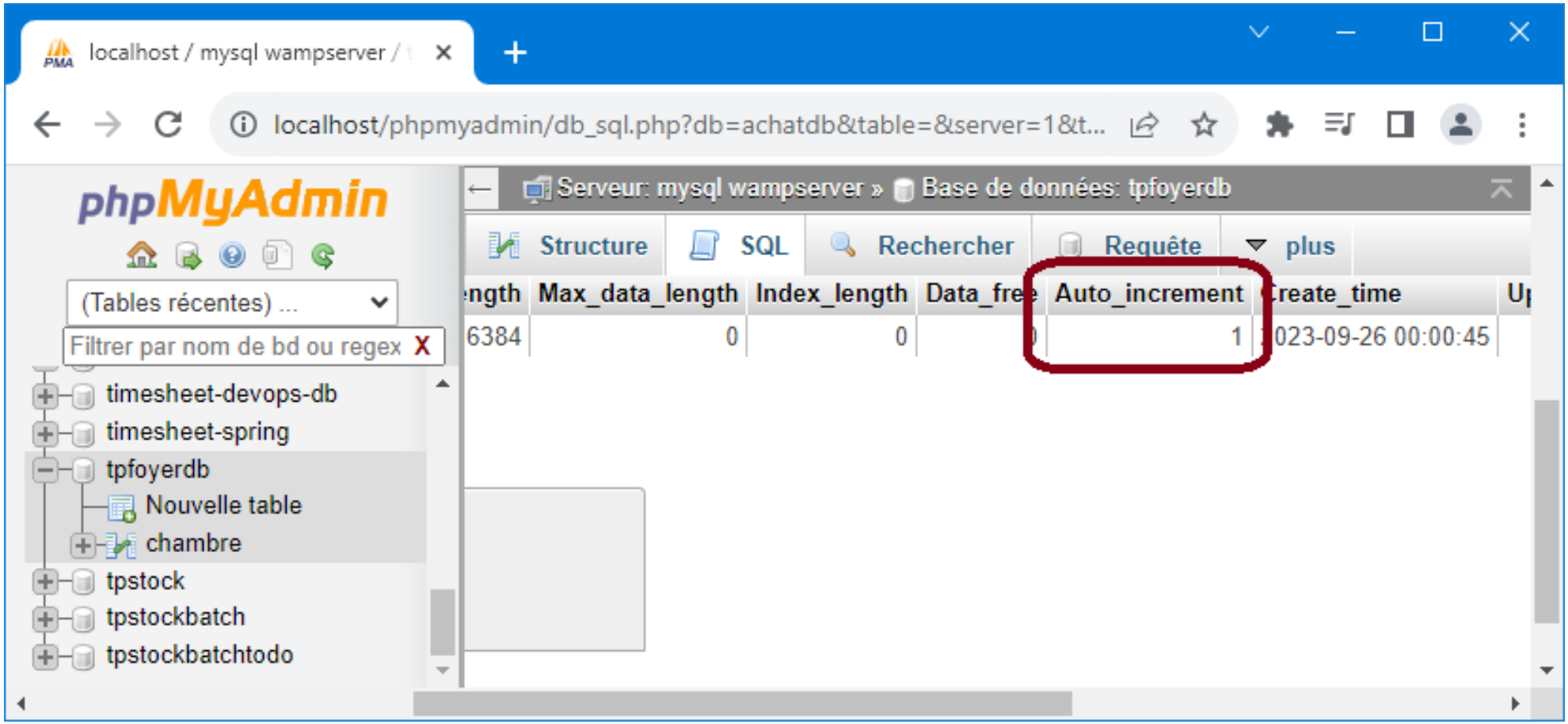
- **Annotation à ajouter dans la classe Chambre :**

```
@Enumerated(EnumType.STRING)
private TypeChambre typeC;
```



Exercice

- Sur **phpmyadmin**, Exécuter la requête SQL : **show table status;**



TP – Spring Data JPA – 1ère Entité

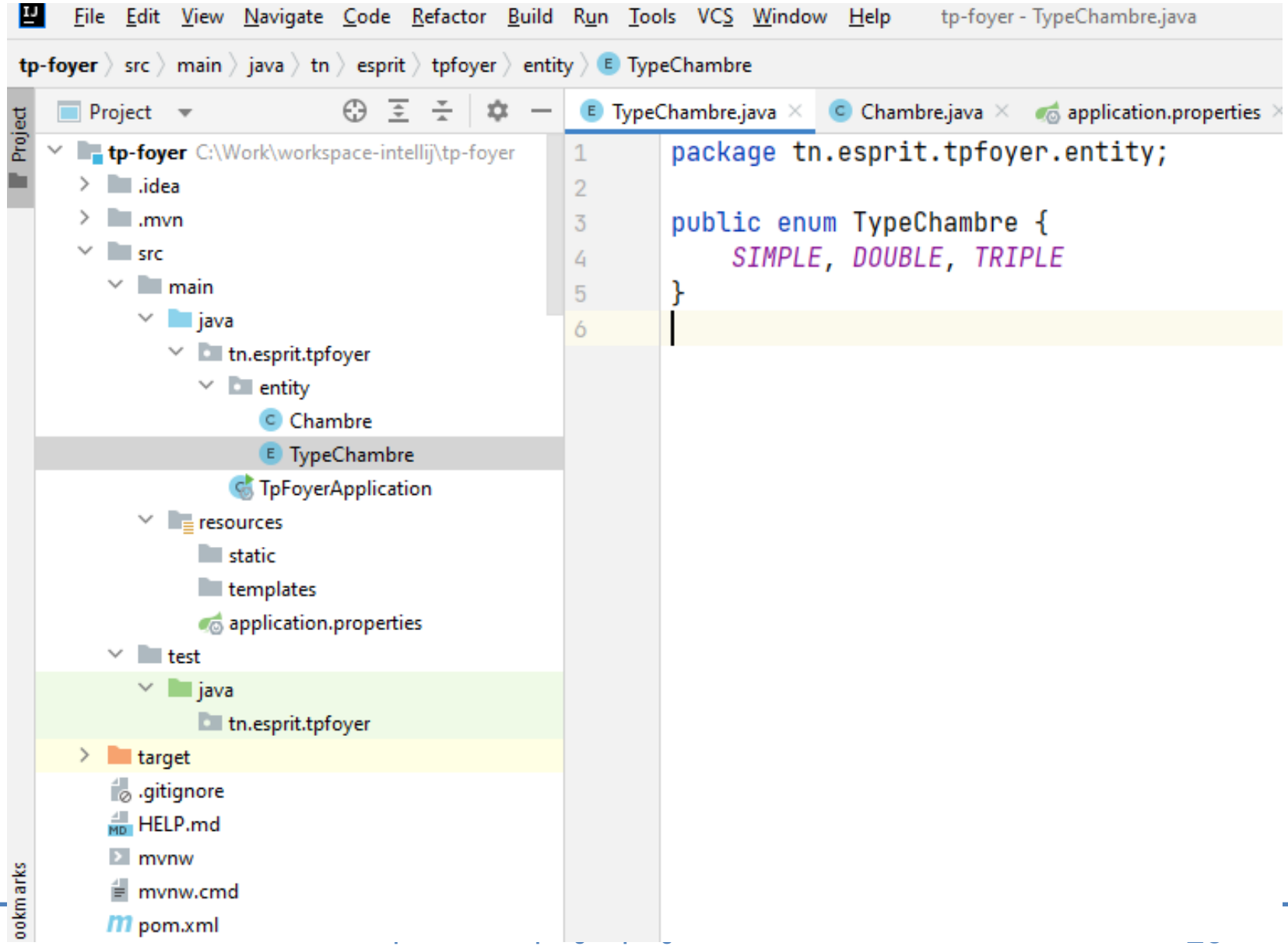
- Nous allons dans ce TP manipuler des données en base de données en utilisant **Spring Data JPA**.
- Pour faciliter l'implémentation de ce TP, nous allons utiliser également **Spring Boot**.
- Les étapes seront décrites dans les slides suivants :

TP – Spring Data JPA – 1ère Entité

- Création d'un projet Spring Boot **tp-foyer** (**Voir Cours 2- Spring Boot**)

Codage

- Coder l'Enum **TypeChambre.java** :



Codage

- Coder l'Entité **Chambre.java** :

File Edit View Navigate Code Refactor Build Run Tools VCS Window Help tp-foyer - Chambre.java

tp-foyer > src > main > java > tn > esprit > tpfoyer > entity > Chambre

Project

- tp-foyer C:\Work\workspace-intellij\tp-foyer
  - .idea
  - .mvn
  - src
    - main
      - java
        - tn.esprit.tpfoyer
          - entity
            - Chambre
            - TypeChambre
          - TpFoyerApplication
      - resources
        - static
        - templates
        - application.properties
    - test
      - java
        - tn.esprit.tpfoyer
  - target
  - .gitignore
  - HELP.md
  - mvnw
  - mvnw.cmd
  - pom.xml

TypeChambre.java | Chambre.java | application.properties

```java
package tn.esprit.tpfoyer.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
public class Chambre {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idChambre;

    private Long numeroChambre;

    @Enumerated(EnumType.STRING)
    private TypeChambre typeC;

}
```

Codage

- Vérifier **application.properties** déjà codé la séance dernière 2- Spring Boot :

File Edit View Navigate Code Refactor Build Run Tools VCS Window Help tp-foyer - application.properties

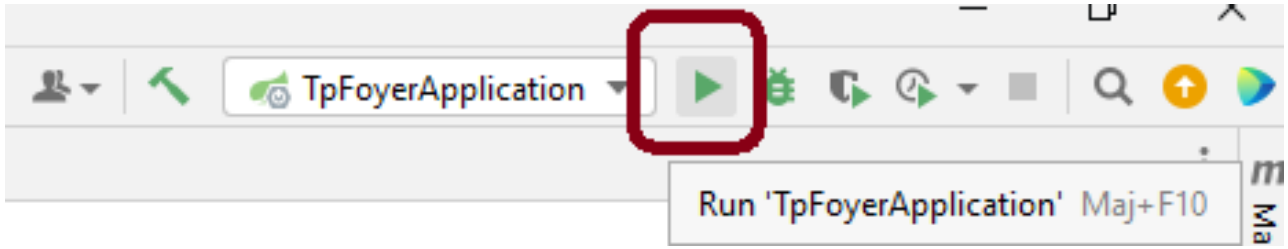
tp-foyer > src > main > resources > application.properties

TypeChambre.java | Chambre.java | application.properties

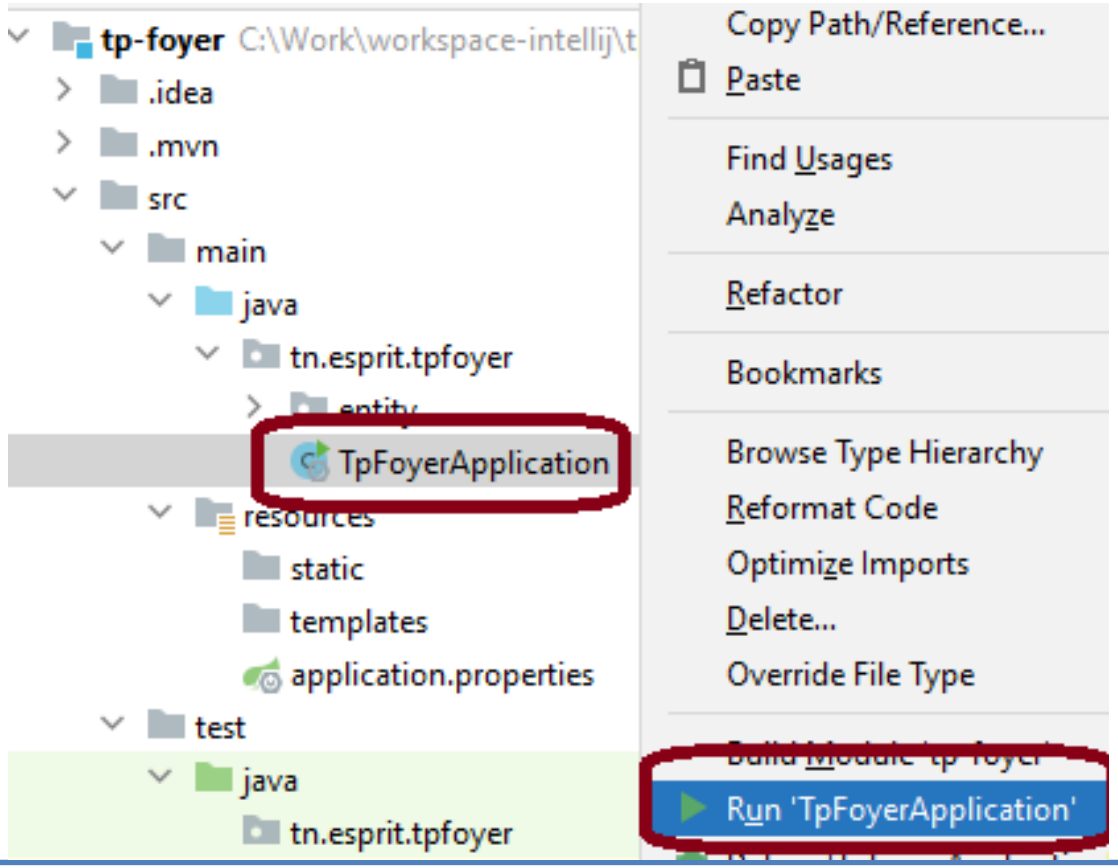
```
spring.datasource.url=jdbc:mysql://localhost:3306/tpfoyerdb?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

Exécution

Lancer le projet Spring Boot « **tp-foyer** »,



Ou :



Exécution

Vérifier qu'il n'y a pas d'erreur. Vérifier que la table a bien été créée dan a base de données :

localhost / mysql wampserver /

localhost/phpmyadmin/index.php?db=achatdb&token=26518255225976f7a43f5395e0d63f2a#PMAURL-8:tbl_structure.php?db=tpfoyerdb&tab

phpMyAdmin

(Tables récentes) ...

Filtrer par nom de bd ou regex

- timesheet-spring
- tpfoyerdb
  - Nouvelle table
  - chambre
    - Colonnes
      - Nouvelle colonne
      - id_chambre
      - numero_chambre
      - typec

Serveur: mysql wampserver » Base de données: tpfoyerdb » Table: chambre

Afficher | Structure | SQL | Rechercher | Insérer | Exporter | Importer | Privilège

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra
1	id_chambre	bigint(20)			Non	*Aucune*	AUTO_INCREMENT
2	numero_chambre	bigint(20)			Oui	*NULL*	
3	typec	enum('DOUBLE', 'SIMPLE', 'TRIPLE')	latin1_swedish_ci		Oui	*NULL*	

Tout cocher *Pour la sélection :* Afficher Modifier Supprimer Primaire Unique Inc

Version imprimable Vue relationnelle Suggérer des optimisations de structure Déplacer des colonnes

Ajouter 1 colonne(s) En fin de table En début de table Après id_chambre

+ Index

Travail À Faire

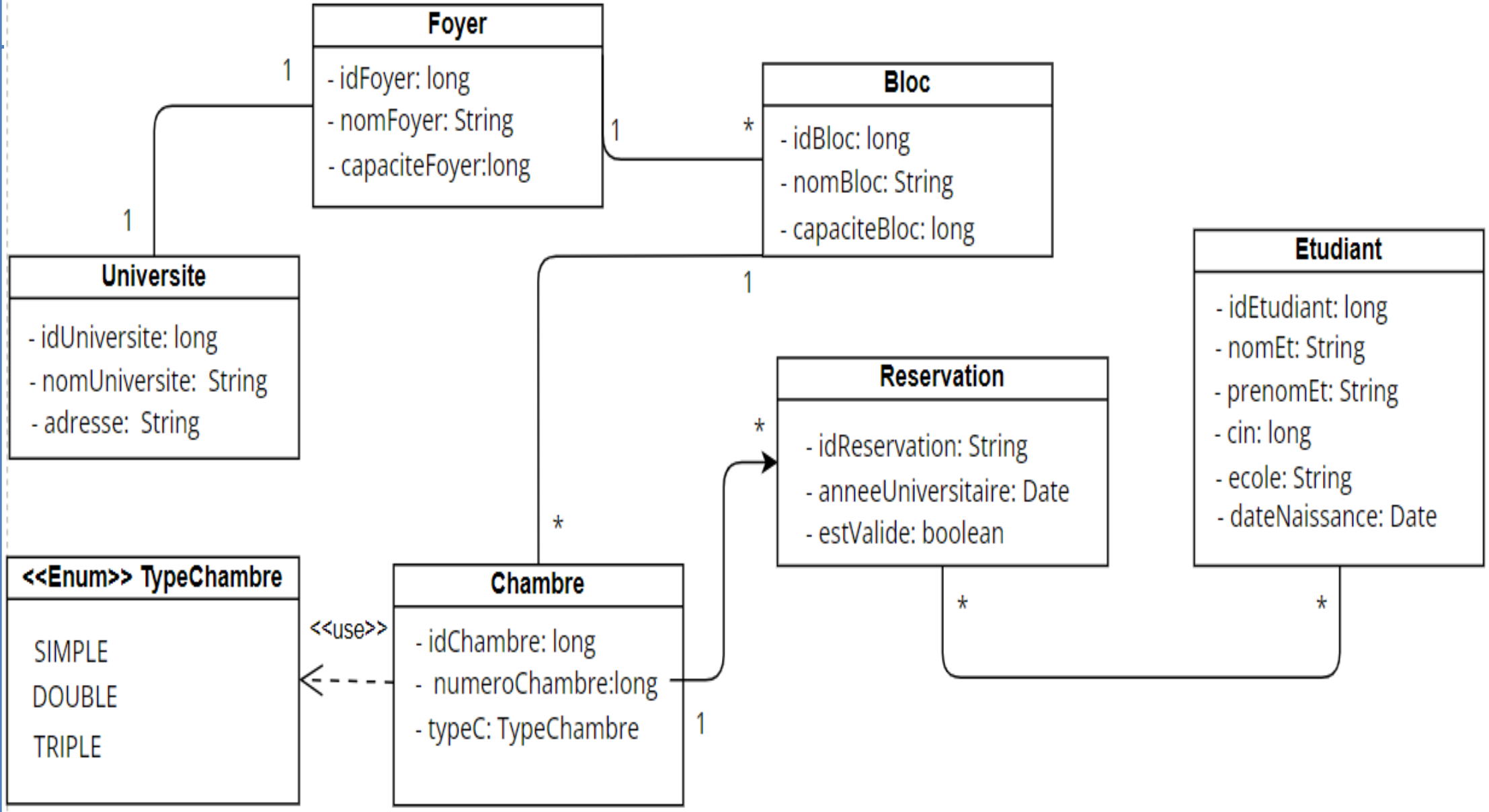
TP étude de cas tp-foyer

Créer les entités se trouvant dans le diagramme des classes **(sans les associations)** et vérifier qu'ils ont été ajoutés avec succès dans la base de données.

Les associations seront vues la prochaines séance.

(Voir diagrammes de classes page suivante)

Diagrammes de Classes



Résultat

File Edit View Navigate Code Refactor Build Run Tool tp-foy

foyer > src > main > TpFoyerApplication

Project

- tp-foyer C:\Work\workspace-intellij\tp-foyer
  - .idea
  - .mvn
  - src
    - main
      - java
        - tn.esprit.tpfoyer
          - entity
            - Bloc
            - Chambre
            - Etudiant
            - Foyer
            - Reservation
            - TypeChambre
            - Universite
          - TpFoyerApplication

Database

- tpfoyerdb@localhost 1 of 30
  - tpfoyerdb
    - tables 6
      - bloc
      - chambre
      - etudiant
      - foyer
      - reservation
      - universite
    - Server Objects

Run: TpFoyerApplication

Console Actuator

```
2023-09-26T00:54:37.399+01:00  INFO 20340 --- [
Hibernate: create table bloc (idb bigint not null au
Hibernate: alter table chambre modify column typec
Hibernate: create table etudiant (ide bigint not nul
Hibernate: create table foyer (idf bigint not null a
Hibernate: create table reservation (id_reservation
Hibernate: create table universite (idu bigint not n
```

Version Control Run TODO Problems Spring Terminal Endpoints

tpfoyerdb@localhost: DBMS: MySQL (ver. 5.6.17) // Cas... (2 minutes ago) 42:1 4 spaces

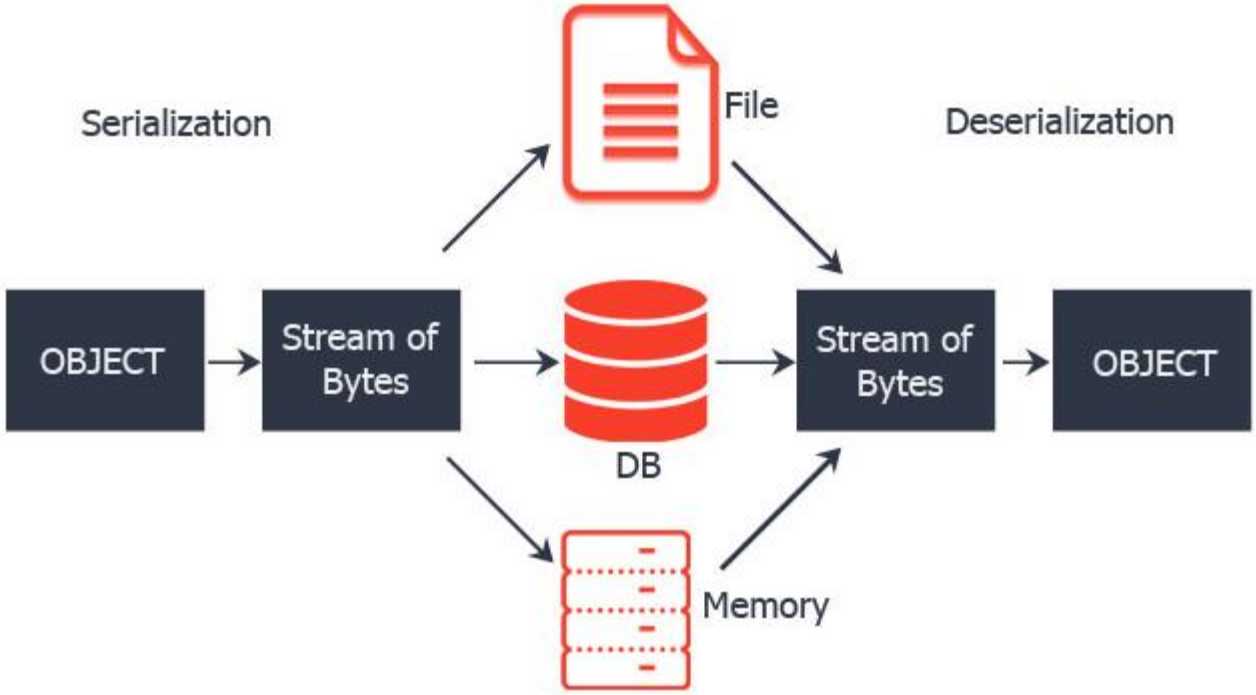
Héritage

- Pour représenter un modèle hiérarchique dans un modèle relationnel, JPA propose alors trois stratégies possibles :

- 1. **Une seule table unique** pour l'ensemble de la hiérarchie des classes. L'ensemble des attributs de toute la hiérarchie des entités est mis à plat et regroupé dans une seule table (il s'agit d'ailleurs de la stratégie par défaut).

- 2. **Une table pour chaque classe concrète**. Chaque entité concrète de la hiérarchie est associée à une table.

- 3. **Jointure entre sous-classes**. Dans cette approche, chaque entité de la hiérarchie, concrète ou abstraite, est associée à sa propre table. Ainsi, nous obtenons dans ce cas là une séparation des attributs spécifiques de la classe fille par rapport à ceux de la classe parente. Il existe alors, une table pour chaque classe fille, plus une table pour la classe parente. Une jonction est alors nécessaire pour instancier la classe fille.

SÉRIALISATION / DÉSÉRIALISATION

- La **sérialisation** est le processus de conversion d'un **objet** en un **flux d'octets** pour stocker l'objet ou le transmettre à la mémoire, à une base de données, ou dans un fichier.
- Son principal objectif est d'enregistrer l'état d'un objet afin de pouvoir le recréer si nécessaire, par mécanisme de **désérialisation**.